

Day 35: Weekly Revision

Day 35 — Weekly Revision

1. Day Number

Day 35

2. Topic Name

Revision of Days 29–34

Today you will revise:

- Dictionary
- List
- Loop
- if-else
- Arithmetic
- f-string formatting

3. Connection

During the last few days, you built small parts of a simple shopping system:

```
Product
  ↓
Product List
  ↓
Add to Cart
  ↓
Calculate Total
  ↓
Apply Discount
  ↓
Print Receipt
```

Today you will combine these ideas into one small program.

The goal is not to learn a completely new Python concept. The goal is to understand **how multiple basic concepts work together**.

4. Revision Summary of Days 29–34

Day 29 — Product Dictionary

You learned how to represent one product using a dictionary.

Conceptually:

```
product = {
    "name": ...,
```

```
"price": ...,  
"stock": ...  
}
```

A dictionary stores information as:

key → value

For example:

name → Pen
price → 20
stock → 5

Day 30 — Product List

You learned that multiple product dictionaries can be stored inside a list.

Concept:

```
products  
├── product 1  
├── product 2  
└── product 3
```

A loop can then process each product.

Day 31 — Add Product to Cart

You learned to create an empty cart:

```
cart = []
```

and use `append()` to add a product.

Before adding the product, you checked its stock.

Conceptually:

```
Is stock available?  
  
↓  
Yes / No  
 /   \  
add  don't  
product  add
```

Day 32 — Calculate Cart Total

You learned the **accumulator pattern**.

Start with:

```
total = 0
```

Then repeatedly add prices while looping through the cart.

Concept:

```
total = 0

product 1 price
  ↓
total increases

product 2 price
  ↓
total increases

Final total
```

Day 33 — Apply Discount

You used if-else to make a decision.

For example:

```
Is total greater than discount limit?
  ↓
  Yes / No
  /   \
discount same
applied total
```

You also revised percentage calculation:

```
discount = total × percentage / 100
```

Day 34 — Print Receipt Using f-string

You learned to place variables inside strings using f-strings.

For example:

```
name = "Notebook"
price = 50

print(f"Product: {name}")
print(f"Price: {price}")
```

This is easier to read than joining many strings manually.

5. Important Topics

Today's main concepts are:

Dictionary

Stores related information about one object.

```
Product
├── name
├── price
└── stock
```

List

Stores multiple items.

```
cart = [product1, product2, ...]
```

Loop

Processes every item in the cart.

```
for each product in cart
    process product
```

If-Else

Makes decisions.

```
if condition is true
    do something
else
    do something else
```

Arithmetic

Used for totals and discounts.

```
price × quantity
total - discount
```

f-string

Used for readable output.

```
print(f"Total: {total}")
```

6. Foundational Notes

Dictionary access

Suppose a dictionary represents a product:

```
product

name  → Mouse
price → 500
stock → 4
```

You access individual values using their keys.

Conceptually:

```
product["name"]
product["price"]
product["stock"]
```

Cart is simply a list

A cart does not need to be complicated.

At this stage:

```
cart = []
```

means:

| The shopping cart currently contains no products.

After adding a product:

```
cart
|
└─ product
```

Check stock before adding

You should not add an unavailable product to the cart.

Think:

```
stock > 0
```

If true:

```
add product
```

Otherwise:

```
do not add product
```

Accumulator pattern

One of the most important beginner patterns is:

```
start value
  ↓
loop
  ↓
add something
  ↓
updated value
```

For cart totals:

```
total = 0

total = total + product price
```

The same pattern appears later in many programs.

Empty cart

An empty cart is valid.

```
cart = []
```

If you loop through an empty list, the loop simply runs **zero times**.

Therefore, if you started with:

```
total = 0
```

the total remains:

```
0
```

Discount should happen after calculating total

Order matters.

Correct thought process:

```
Build cart
  ↓
Calculate total
  ↓
Check discount
  ↓
Calculate final total
  ↓
Print receipt
```

You cannot correctly calculate a discount before knowing the total.

7. Easy Example

Imagine a product:

```
Name  : Notebook
Price : ₹100
Stock : 3
```

Stock is greater than 0, so it can be added to the cart.

If the cart contains only this product:

```
Cart
└─ Notebook ₹100
```

Then:

```
Starting total = 0

total = 0 + 100

Final total = 100
```

If the discount rule requires a total greater than ₹500:

```
100 > 500 ?
```

No.

Therefore:

```
Discount = 0
Final total = ₹100
```

A receipt could conceptually show:

```
----- RECEIPT -----
Product: Notebook
Total: ₹100
Discount: ₹0
Final Total: ₹100
-----
```

This is only an illustration of the expected flow, not the complete solution.

8. Revision Problem Statement

Create a beginner-friendly Python program that:

1. Creates **one product dictionary** containing:

- product name
- price
- stock

2. Creates an **empty cart list**.

3. Checks whether the product stock is greater than 0.

4. If stock is available:

- add the product to the cart.

5. Use a loop to calculate the cart total.

6. Apply a simple discount rule such as:

```
If total > 500
    apply 10% discount
Otherwise
    keep the same total
```

7. Print a simple receipt using f-strings.

Do not worry about building a real shopping application yet.

The purpose is to combine the concepts from Days 29–34.

9. Concepts Used

You will use:

```
Dictionary
  ↓
Store product information

List
  ↓
Store cart products

if
  ↓
Check stock

append()
  ↓
Add product to cart

for loop
  ↓
Process cart

Accumulator
  ↓
Calculate total

Arithmetic
  ↓
```



```
Calculate discount
```

```
if-else
```

```
↓
```

```
Decide whether discount applies
```

```
f-string
```

```
↓
```

```
Print receipt
```

10. Thought Process

Before writing Python, break the problem into small steps.

Step 1 — Represent the product

Ask:

| What information belongs to the product?

You need:

```
name  
price  
stock
```

A dictionary is suitable.

Step 2 — Create the cart

Start with an empty list.

Think:

```
cart initially contains nothing
```

Step 3 — Check availability

Before adding the product:

```
Is stock greater than 0?
```

If yes:

```
add it
```

If no:

```
leave cart empty
```

Step 4 — Calculate total

Start:

```
total = 0
```

Loop through the cart.

For every product:

```
add its price to total
```

Step 5 — Decide discount

Ask:

```
Is total greater than the discount limit?
```

If yes:

```
calculate discount
```

Otherwise:

```
discount = 0
```

Step 6 — Calculate final amount

Conceptually:

```
final amount = total - discount
```

Step 7 — Print receipt

Use f-strings to display useful values such as:

```
product name  
total  
discount  
final total
```

The overall thinking is:

```
Product  
↓  
Check stock  
↓  
Add to cart  
↓  
Calculate total  
↓  
Check discount
```

```
↓  
Final amount  
↓  
Receipt
```

11. Pseudocode

```
START  
  
CREATE a product containing  
    name  
    price  
    stock  
  
CREATE an empty cart  
  
IF product stock is greater than 0  
    ADD product to cart  
ELSE  
    display that product is unavailable  
  
SET total to 0  
  
FOR each product in cart  
    ADD product price to total  
  
IF total is greater than 500  
    CALCULATE 10 percent discount  
ELSE  
    SET discount to 0  
  
CALCULATE final total  
  
PRINT receipt using f-strings  
  
END
```

Notice that pseudocode describes the **logic**, not exact Python syntax.

12. Suggested Solving Approach — Functional Approach

Use small functions where each function has one simple responsibility.

Conceptually:

```
main program  
|  
├─ create_product()  
|  
├─ add_to_cart()  
|
```

```
|— calculate_total()
|— apply_discount()
|— print_receipt()
```

You do not necessarily need all five functions if that feels too complicated.

A simpler beginner structure could be:

```
calculate_total(cart)
    ↓
returns total

apply_discount(total)
    ↓
returns final amount
```

Why use functions?

Without functions, all the logic can become one long block.

With functions:

```
input/data
    ↓
function
    ↓
result
```

Each part becomes easier to understand and test.

For example:

```
cart
    ↓
calculate_total()
    ↓
total
```

Then:

```
total
    ↓
apply_discount()
    ↓
final total
```

This prepares you for larger programs later.

13. Easy Edge Cases

Edge Case 1 — Stock is 0

Example:

```
name = Keyboard  
price = 700  
stock = 0
```

The product should **not** be added to the cart.

Expected idea:

```
cart = empty
```

Therefore the total should remain:

```
0
```

Edge Case 2 — Empty Cart

Suppose:

```
cart = []
```

Your loop should still work correctly.

Because there are no products:

```
total = 0
```

This is why initializing the accumulator is important.

Edge Case 3 — Total Below Discount Limit

Suppose:

```
total = 300
```

and discount requires:

```
total > 500
```

Then:

```
discount = 0  
final total = 300
```

Bonus Edge Case — Total Exactly 500

Pay attention to the condition.

If the requirement says:

```
greater than 500
```

then:

```
500 > 500
```

is false.

Therefore ₹500 should receive **no discount**.

This distinction is important:

```
> 500    means more than 500
```

```
>= 500   means 500 or more
```

14. Common Mistakes to Avoid

Mistake 1 — Using the wrong dictionary key

For example, creating:

```
"price"
```

but later trying to access:

```
"Price"
```

Dictionary keys are case-sensitive.

Mistake 2 — Forgetting to create the cart

You need a list before using `append()`.

Conceptually:

```
create cart  
  ↓  
append product
```

Mistake 3 — Adding product without checking stock

Avoid:

```
product → cart
```

without checking availability.

Instead:

```
product
  ↓
check stock
  ↓
cart
```

Mistake 4 — Forgetting total = 0

Before accumulating values, initialize the accumulator.

```
total = 0
```

Otherwise Python will not know the starting value.

Mistake 5 — Replacing instead of accumulating

Wrong idea:

```
total = current_product_price
```

inside every loop iteration.

That would repeatedly replace the previous total.

You need the idea:

```
old total + current price
      ↓
new total
```

Mistake 6 — Incorrect percentage calculation

Remember:

```
10% of total
```

means:

```
total × 10 / 100
```

not simply:

```
total - 10
```

Mistake 7 — Checking discount before calculating total

Keep the order:

```
calculate total
  ↓
check discount
```

not the other way around.

Mistake 8 — Forgetting the f

This:

```
print("Total: {total}")
```

does not insert the variable.

An f-string starts with:

```
f"..."
```

15. Quick Self-Check Questions

Question 1

Which Python data structure is suitable for storing:

```
name
price
stock
```

for one product?

Think: list or dictionary?

Question 2

Why do we write something like:

```
total = 0
```

before calculating the cart total?

Question 3

If:

```
stock = 0
```

should the product be added to the cart?

Question 4

If the discount condition is:


```
total > 500
```

will a total of exactly 500 receive the discount?

Question 5

What is the purpose of the `f` in:

```
print(f"Total: {total}")
```

Try answering these without looking back at the notes.

16. Hint Only

Build the program in the same order that a shopping process happens:

```
Create product
    ↓
Create empty cart
    ↓
Check stock
    ↓
Add available product
    ↓
Start total at 0
    ↓
Loop through cart
    ↓
Add prices to total
    ↓
Check discount condition
    ↓
Calculate final total
    ↓
Print receipt with f-strings
```

For the functional approach, think about which values should move between functions:

```
cart
    ↓
calculate_total()
    ↓
total
    ↓
apply_discount()
    ↓
final total
```

Your main challenge for **Day 35** is not Python syntax—it is connecting the six concepts correctly:

dictionary → list → if-else → loop → arithmetic → f-string.